

Sparse Exploratory Approximate Factor Analysis: A Tutorial for the R package seafar

Tra Le¹, Stefan Kirsch², and Katrijn Van Deun¹

¹Department of Methodology and Statistics, Tilburg University

²Research Service Department, Tilburg University

Author Note

Tra Le  <https://orcid.org/0000-0002-7700-8946>

Stefan Kirsch  <https://orcid.org/0009-0009-6939-411X>

Katrijn Van Deun  <https://orcid.org/0000-0002-2271-793X>

The authors have no conflicts of interest to declare that are relevant to the content of this article. This publication is part of the project SEM2.0 (with project number NWO 406.22.GO.022) of the Open Competition research program which is financed by the Dutch Research Council (NWO).

Correspondence concerning this article should be addressed to Tra Le, Email: t.t.le_1@tilburguniversity.edu

Abstract

This manuscript provides a tutorial for the package `seafar` in the statistical software R to perform a Sparse Exploratory Approximate Factor Analysis for high-dimensional data. The methods used in this package make use of the approximate factor model together with cardinality constraint. This helps deal with exploratory high-dimensional settings and complex models characterized by many parameters for a limited sample size, in which most latent variable methods fail. The objectives of this package are two-fold: i) provide users with easy control over the exact number of nonzero/zero loadings for the simple structure of the factor loadings; ii) model selection procedure to help inform users about the simple structure in an exploratory setting with little to no prior information; iii) extract factor scores which can be used in further analyses. The use of `seafar` package is demonstrated using two empirical data sets.

Keywords: exploratory factor analysis, high-dimensional data, tutorial, R package

Sparse Exploratory Approximate Factor Analysis: A Tutorial for the R package *seafar*

Studying latent constructs, such as personality, intelligence, and well-being, is at the core of behavioral research. However, it is met with new challenges introduced by the “Big Data” era, characterized by intensive data collection and technologically advanced measurement tools. Specifically, researchers often need to deal with data sets that have more variables than the number of observations, or so-called high-dimensional low-sample-size (HDLSS) data. Furthermore, new types of data (e.g., GPS or physiological data collected using smart watches) are often exploratory in nature with limited or no prior theoretical knowledge, which can lead to complex models with many unknown parameters relative to the number of observations. Thus, existing (exploratory) latent variable methods often fail to work in these settings, yielding unstable and/or improper solutions or having non-convergent issues. We refrain from discussing in detail these methods and their issues in this paper; however, interested readers are referred to Le et al. (2026) for comparative overviews.

Regularized ESEM (Le et al., 2026) was proposed to solve the abovementioned issues. This two-stage method relies on the approximate factor analysis framework [citation] to extract latent factors underlying high-dimensional data. In the first stage, Regularized ESEM imposes simple structure on the loading matrix through cardinality constraint for orthogonal factors and the LASSO penalty for the non-orthogonal factors. The authors proposed a sequential model selection strategy to tune the algorithm: first, the number of factors is determined, after which the number of nonzero loadings (cardinality value or LASSO penalty parameter) is chosen to maximize the Index of Sparseness (Gajjar et al., 2017; Gu et al., 2019; Trendafilov, 2014). The current manuscript introduces an R package called *seafar* to implement this first stage with a few modifications. Firstly, instead of having two types of regularization (i.e., cardinality constraint and LASSO penalty) separately for orthogonal and non-orthogonal factors, *seafar* implements a unified algorithm that uses cardinality constraint for both orthogonal and non-orthogonal factors. Secondly, *seafar* offers the original model selection procedure with the Index of Sparseness and an alternative procedure using warm starts (Friedman et al., 2010) which can help reduce the

computational time.

The rest of the paper is structured as follows: first, we introduce the Sparse Exploratory Approximate Factor Analysis (SEAFa) model and its model selection procedures; next, the use of the seafar package is demonstrated using two empirical data sets. The paper is concluded with some remarks.

Method

In this section, we will introduce the notation used in this paper, the SEAFa model, its estimation, and model selection procedures.

Data and notation

Here, we introduce the notation used throughout this paper. Matrices are denoted by bold uppercase letters and vectors by bold lowercase letters. Transposes are indicated by the superscript T , and scalars are denoted by lowercase italic letters. For a matrix $\mathbf{A}$, $\text{Card}(\mathbf{A})$ denotes the number of nonzero elements in $\mathbf{A}$.

Let $\mathbf{Y} \in \mathbb{R}^{N \times J}$ be the data matrix of interest consisting of J standardized items (i.e., items are mean-centered and scaled to unit variance) for N observations. The squared Frobenius norm of $\mathbf{Y}$ is denoted as $\|\mathbf{Y}\|_2^2 = \sum_{i=1}^N \sum_{j=1}^J y_{ij}^2$.

The SEAFa model

It is important to point out that the SEAFa model is essentially the measurement model of Regularized ESEM [Le2026exploratory]. Here, we outline the details of the model again. Let $\mathbf{y}$ be a $J \times 1$ vector of J standardized items, $\mathbf{P}$ be a $J \times Q$ matrix of factor loading matrix, and $\boldsymbol{\eta}$ be a $Q \times 1$ vector of factor scores, we have the SEAFa model as follows:

$$\mathbf{y} = \mathbf{P}\boldsymbol{\eta} + \boldsymbol{\epsilon} \quad (1)$$

where $\boldsymbol{\epsilon}$ is the $J \times 1$ vector of residuals. Model (1) is subject to two identification constraints: 1) $\mathbf{P}$ is assumed to show simple structure (i.e., not all items load on all factors but some/many have zero loadings), and 2) factor scores $\boldsymbol{\eta}$ are subject to length restriction (i.e., each factor η_q is restricted to

have the same length equal to $\sqrt{N}$). Furthermore, the residuals ϵ are assumed to have mean zero and to be uncorrelated with the factor scores. Here, ϵ are allowed to be weakly correlated, in contrast to the common factor model where residuals are assumed to be uncorrelated.

As discussed in @le2026exploratory, a least-squares approach is used to estimate the approximate factor model in (1), as it has favorable properties in high-dimensional settings @fan2021robust. Let p_{jq} denote the loading of variable j on factor q and η_{iq} denote the score of person i on factor q , the least-squares objective function is as follows:

$$\begin{aligned} & \arg \min_{\eta, p} \sum_{j=1}^J \sum_{i=1}^N (y_{ij} - \sum_q \eta_{iq} p_{jq})^2 \\ & \text{subject to } \sum_i \eta_{iq}^2 = N \quad \forall q = 1, \dots, Q, \end{aligned} \quad (2)$$

and $\mathbf{P}$ showing simple structure.

Regularized ESEM [@le2026exploratory] chose to obtain simple structure for $\mathbf{P}$ in two ways: i) imposing a hard constraint on $\mathbf{P}$ to have exactly C nonzero loadings (for orthogonal factors); ii) using the LASSO penalty to shrink (many) loadings toward zeros (for non-orthogonal factors). Here, SEAFA unifies these approaches by enforcing simple structure solely through a cardinality constraint on $\mathbf{P}$, for both orthogonal and non-orthogonal factors. Thus, the objective function of SEAFA is as follows:

$$\begin{aligned} & \arg \min_{\eta, p} \sum_{j=1}^J \sum_{i=1}^N (y_{ij} - \sum_q \eta_{iq} p_{jq})^2 \\ & \text{subject to } \sum_i \eta_{iq}^2 = N \quad \forall q = 1, \dots, Q, \end{aligned} \quad (3)$$

and $\text{Card}(\mathbf{P}) = C$,

or in matrix form

$$\begin{aligned} & \arg \min_{\mathbf{H}, \mathbf{P}} \|\mathbf{Y} - \mathbf{H}\mathbf{P}^T\|_F^2, \\ & \text{subject to } \boldsymbol{\eta}_q^T \boldsymbol{\eta}_q = N \quad \forall q = 1, \dots, Q, \\ & \text{and } \text{Card}(\mathbf{P}) = C. \end{aligned} \quad (4)$$

To solve (4), we use an alternating optimization (AO) scheme: factor scores $\mathbf{H}$ and loadings $\mathbf{P}$ are updated alternately while holding the other fixed. Specifically, with fixed $\mathbf{P}$, $\hat{\mathbf{H}}$ (subject to norm constraint) is obtained using the method of Lagrange multipliers. $\hat{\mathbf{P}}$ is estimated with fixed $\mathbf{H}$ via the cardinality-constrained linear regression problem:

$$\begin{aligned} \hat{\mathbf{P}} &= \arg \min_{\mathbf{P}} \|\text{vec}(\mathbf{Y}) - (\mathbf{I} \otimes \mathbf{H})\text{vec}(\mathbf{P}^T)\|_2^2, \\ & \text{subject to } \text{Card}(\mathbf{P}) = C, \end{aligned} \quad (5)$$

where $\otimes$ is the Kronecker product. $\hat{\mathbf{P}}$ is updated using a numerical solution proposed by @adachi2017sparse as:

$$\mathbf{P}_{new} = \mathbf{B} \text{ whose elements, except for the } C \text{ largest (in absolute value), are set to zero,} \quad (6)$$

where $\mathbf{B} = \mathbf{P}_{old} - \alpha^{-1}(\mathbf{P}_{old}\mathbf{H}^T\mathbf{H} - \mathbf{Y}^T\mathbf{H})$ with α being the largest eigenvalue of $\mathbf{H}^T\mathbf{H}$. The detailed derivation of $\mathbf{B}$ can be found in Appendix A. This update can be seen as a solution to a projected gradient scheme with fixed step size α^{-1} (Guerra-Urzola et al., 2023). In the special case of orthogonal factors, that is, when $\mathbf{H}^T\mathbf{H} = N\mathbf{I}$, @adachi2016sparse proposed a computationally efficient update of $\mathbf{H}$ and $\mathbf{P}$ as follows: given $\mathbf{P}$, $\hat{\mathbf{H}}$ is the solution to the Orthogonal Procrustes problem; Given $\mathbf{H}$, $\hat{\mathbf{P}}$ is updated in a similar manner as in (6), but with $\mathbf{B} = N^{-1}\mathbf{Y}^T\mathbf{H}$.

The AO procedure results in a non-increasing sequence of loss values and converges to a stationary point. However, as (4) is not a convex problem, the algorithm is subject to local minima.

Thus, we implement the algorithm with a multi-start procedure: the algorithm is run several times with different starting values and solution with the lowest value is retained as the final solution.

The implementation of SEAFA is summarized in Algorithm 1.

Algorithm 1: Sparse Exploratory Approximate Factor Analysis (SEAFA)

Input: $\mathbf{Y}, Q, C, \text{MaxIter}, \epsilon$

Output: $\hat{\mathbf{P}}, \hat{\mathbf{H}}$

Initialize loading matrix $\mathbf{P}^{(0)}$ having $*C*$ nonzero loadings

```

while  $\Delta \text{lossfunction} > \epsilon$  or  $\text{iteration} \neq \text{MaxIter}$  do
  if  $\eta_q$  are orthogonal then
     $\mathbf{H} \leftarrow \sqrt{N} \mathbf{U} \mathbf{V}$  from the SVD of  $N^{-1/2} \mathbf{Y} \mathbf{P} = \mathbf{U} \mathbf{S} \mathbf{V}^T$ 
     $\mathbf{A} \leftarrow N^{-1} \mathbf{Y}^T \mathbf{H}$ 
  else
    while  $\Delta \text{lossfunction2} > \epsilon$  or  $\text{iteration} \neq \text{MaxIter}$  do
      for  $q \leftarrow 1$  to  $Q$  do
         $\mathbf{E}_q \leftarrow \mathbf{Y} - \mathbf{H} \mathbf{P}_q^T + \eta_q \mathbf{p}_q^T$ 
         $\eta_q \leftarrow \frac{\sqrt{N} \mathbf{E}_q \mathbf{p}_q}{\sqrt{(\mathbf{E}_q \mathbf{p}_q)^T (\mathbf{E}_q \mathbf{p}_q)}}$ 
      end
    end
     $\alpha \leftarrow$  maximum eigenvalue of  $\mathbf{H}^T \mathbf{H}$ 
     $\mathbf{A} = (\mathbf{P}^T - \frac{\mathbf{H}^T \mathbf{H} \mathbf{P}^T - \mathbf{H}^T \mathbf{Y}}{\alpha N})^T$ 
  end
  for  $q \leftarrow 1$  to  $Q$  do
     $\mathbf{p}_q := T_{C_q}(\mathbf{a}_q)$ 
  end
end
return  $\hat{\mathbf{P}}, \hat{\mathbf{H}}$ 

```

Model selection

Algorithm 1 requires five inputs: a matrix of standardized items $\mathbf{Y}$, number of factors Q , number of nonzero loadings C , and two stopping criteria. While all inputs can be user-specified, the number of factors and nonzero loadings may be unknown beforehand. Thus, we propose a sequential data-driven strategy similar to Le et al. (2026) to choose these two inputs:

- Step 1: Number of factors Q can be specified either based on prior knowledge or by using different factor retention methods such as parallel analysis (Horn, 1965), scree plot

(Cattell, 1966), Kaiser-Guttman rule (Kaiser, 1960), or a machine learning method (Goretzko & Bühner, 2020).

- Step 2: Based on the selected number of factors, the number of nonzero loadings C is chosen using the Index of Sparseness (Gajjar et al., 2017; Trendafilov, 2014). That is, the algorithm searches through a sequence of candidate values for C and chooses the one that maximizes the Index of Sparseness (IS), which is defined as follows:

$$IS = PEV_{PCA} \times PEV_{rESEM} \times PS, \quad (7)$$

where PEV_{PCA} , PEV_{SEAFa} and PS are the proportion of explained variance using ordinary PCA, the proportion of explained variance using SEAFa, and the proportion of zero loadings, respectively. The IS value increases with increasing PEV_{PCA} , increasing PEV_{SEAFa} , and the proportion of zero loadings. Thus, IS balances goodness-of-fit with the complexity of the solution (sparser solutions being less complex). A suitable range of C is $Q \times 3$ (at least three nonzero loading per factor) to $J \times Q$.

Note that here in Step 2, PEV_{SEAFa} is obtained by fitting SEAFa for each candidate value of C using multiple starting values. This procedure can become computationally intensive when the number of variables J is large, as the candidate range for C increases accordingly. To mitigate this issue, we propose an alternative strategy based on warm starts (Friedman et al., 2010).

Specifically, for a decreasing sequence of candidate values of C , the loading matrix obtained for a given C is used as the starting value for the subsequent (smaller) value of C , while computing the IS at each step. This can help speed up the computational time substantially; however, the results are more dependent on the initial solution, as subsequent estimates build on the loading matrix obtained in the first iteration.

The seafar package

In this section, we demonstrate the use of our package in R using two empirical datasets. The first analysis focuses on recovering the loading patterns of the Big Five dataset

(low-dimensional scenario), while the second analysis explores the factor scores of the Autism Genetic dataset from Nishimura et al. (2007) (ultra-high-dimensional scenario). **First, install and load the *seafar* package as follows:**

```
# Install devtools if needed
if (!requireNamespace("devtools", quietly = TRUE)) {
  install.packages("devtools")
}
devtools::install_github("trale97/seafar")
library(seafar)
```

Analysis 1: Factor loading structure of the Big Five

Data pre-processing

The Big Five dataset can be obtained directly from the qgraph package in R (Epskamp et al., 2012). The dataset contains scores of 500 observations on the 240 items of the personality questionnaire. All functions in seafar work with standardized items, thus, each variable is standardized to have a mean of zero and unit variance before the analysis:

```
data("big5", package = "qgraph")
big5_std <- scale(big5, center = TRUE, scale = TRUE)
```

Number of factors

In line with the design of the questionnaire used to collect the data, we assume five factors in correspondence with the five personality traits. Alternatively, one can use different data-driven methods to extract number of factors by using the `factor_retention()` function in the seafar package. This function calls on existing R implementations of the three factor retention methods, namely parallel analysis from package psych (William Revelle, 2026) and PCAtools (Blighe et al., 2026), scree plot from PCAtools, and Kaiser-Guttman rule from EFA.dimensions (O'Connor, 2026).. The results can be inspected by calling `summary()` as below:

```
library(seafar)
Q <- factor_retention(data = big5_std)
summary(Q)
```

Number of factors by parallel analysis is: 18

Number of factors by Kaiser rule is: 12

Number of components by scree test is: 5

Analysis with known cardinality

Given the design of the questionnaire (with each of the items constructed to measure one trait), we first focus on a solution with 240 nonzero loadings and the remaining 960 zero. We perform a SEAFA analysis using a preset cardinality of 240 with a multi-start procedure:

```
big5_multistart <- seafar_multistart(
  data = big5_std,
  nfactors = 5,
  C = 240,
  INIT = 'rational'
)
```

Note that here, we did not restrict the five factors to be orthogonal. Users can specify `orthogonal = TRUE` to impose orthogonality on the factors. The default number of starting values is 50, which can be modified by option `nstarts =`. Users can also specify the number of nonzero loadings per factor instead of across all factors. Since the questionnaire was designed to measure each trait with 48 items, we indicate a cardinality of 48 nonzero loadings per factor by changing the input `C = rep(48,5)`. Furthermore, users can choose the method to initialize the loading matrix by specifying the option `INIT =` with the following choices:

- `INIT = mixed`: the initial loading matrix is a mixed between the random starts (30%) and starting values based on the Singular Value Decomposition (SVD) of the items (70%). **This is the default option when nothing is specified.**

- `INIT = rational`: the initialization is inspired by the approach by Ding and He (2004) where principal components can be interpreted as continuous relaxations of cluster membership indicators. Here, for the initialization of $\mathbf{P}$, an SVD-based loading matrix is first computed, which are then used to cluster the variables into Q clusters. The resulting cluster structure is then used as a target for rotation, yielding a loading matrix with a simple structure with no cross-loadings. This choice is motivated by the common assumption in questionnaire data that items primarily load on a single latent factor with no cross-loadings (NEEDS CITATION). The resulting rotated loading matrix is used as the initial loading matrix for SEAFA.
- `INIT = random`: the initial loading matrix have complete random values sampled from a multivariate normal distribution $MVN(\mathbf{0}, \mathbf{I})$.

The results can be summarized using `'summary()'` function with option `'disp = '` with two values:

- `disp = "loadings"` only prints the first ten rows of the loading matrix.
- `disp = "full"` prints both the factor score and loading matrices.

```
summary(big5_multistart, disp = "loadings")
```

The number of nonzero loadings is: 240

The estimated loadings matrix is

	[,1]	[,2]	[,3]	[,4]	[,5]
N1	0	0	0	0.569	0
E2	0	0.366	0	0	0
O3	0	0	0.592	0	0
A4	0	0.462	0	-0.313	0
C5	0	0	0	0	0.464

N6	0	0	0	0.478	0
E7	0.453	0	0	0	0
O8	0	0	0.536	0	0
A9	0	0.503	0	0	0
C10	0	0	0	0	0.238

The full factor score and loading matrices can be extracted by calling `big5_multistart$scores` and `big5_multistart$loadings`, respectively. This is useful when users wish to use the full factor scores and/or loadings for further analyses, for instance, plot a heat map for the loadings.

As can be seen, with the rational starting values for the loading matrix, each item only loads on one factor and there is no cross-loading. Table 1 shows the number of nonzero loadings per factor corresponding for each of the five personality traits.

Table 1

The number of nonzero loadings per factor corresponding to each personality trait

Traint	Factor 1	Factor 2	Factor 3	Factor 4	Factor 5
Openness	30	2	0	2	1
Conscientiousness	0	42	6	2	5
Extraversion	3	1	33	10	6
Agreeableness	2	0	7	35	2
Neuroticism	1	3	5	4	38

Note. The factors were ordered in such that the largest number of nonzero loadings is on the diagonal.

The above analyses were done using a multi-start procedure. The package also offers a single-start procedure by using the `seafar()` function as follows:

```
big5_result <- seafar(
  data = big5_std,
  nfactors = 5,
  C = 240,
  INIT = "svd"
)
```

Here, the `INIT = input` offers an extra options for starting values in addition to the starting configurations above:

- `INIT = svd`: for starting values based on the Singular Value Decomposition of the items.

The single-start procedure is computationally faster than the multi-start one, especially when the items matrix is large. However, it does not guarantee to work well in all scenarios but rather is heavily dependent on the type of input data itself. Users are advised to use it with caution.

Analysis with unknown cardinality

Often in an exploratory setting, the number of nonzero loadings is not known beforehand. Here, users can use the `is.seafar()` function for model selection based on the Index of Sparseness. The function requires the number of factors and based on this, it goes through a sequence of 100 possible values for the cardinality and picks the one that results in the highest Index of Sparseness (Gu et al., 2019).

```
model_selection <- is.seafar(  
  data = big5_std,  
  nfactors = 5,  
  INIT = "rational",  
  nstarts = 10,  
  TOL = 3,  
  THR = .2  
)
```

When the dataset is large, this model selection procedure can be computationally intensive. Thus, users can use the `is.seafar_wst()` function that makes use of warm-starts as implemented by the `glmnet` package (Friedman et al., 2010) to speed up the computation time. This function also allows users to specify the number of candidate values to be tested within this range; here, we chose 200 values.

```
model_selection_wst <- is.seafar_wst(  
  data = big5_std,  
  nfactors = 5,  
  INIT = "rational",  
  card.length = 200  
)  
summary(model_selection_wst)
```

The cardinality selected by the Index of Sparseness is: 247

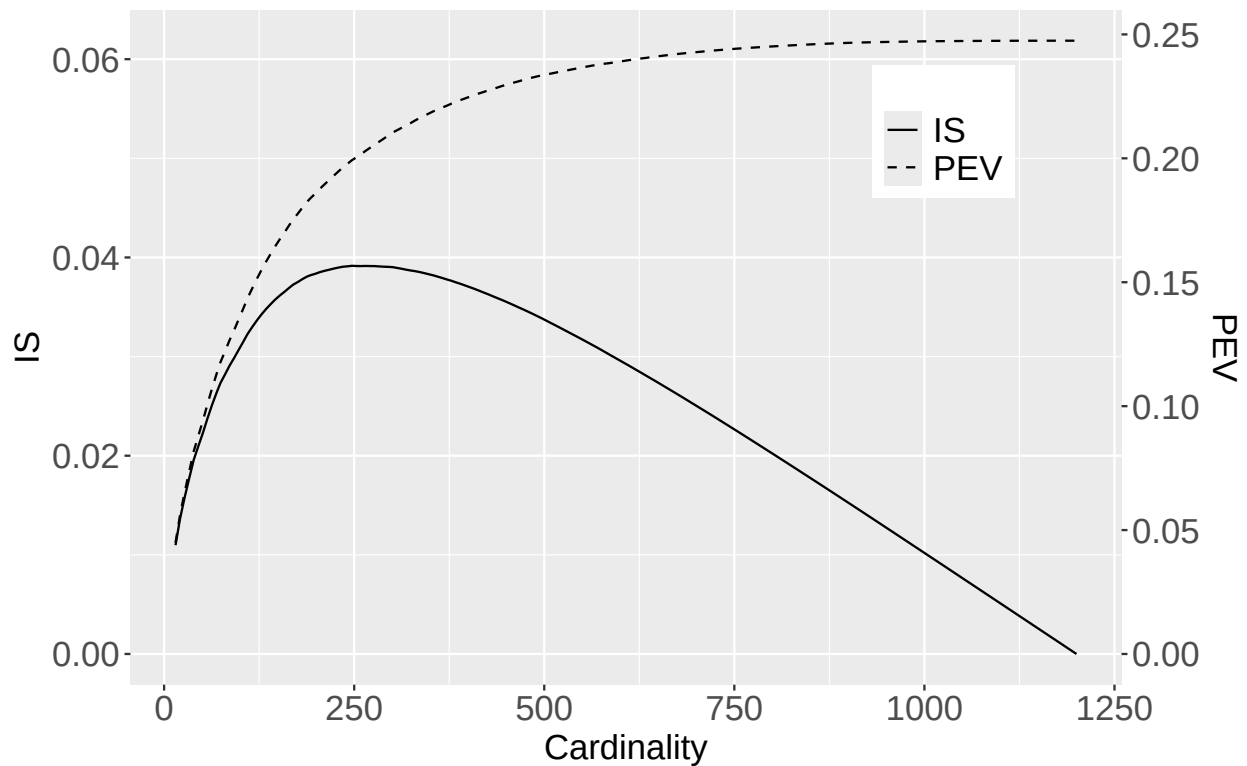
The PEV with this cardinality is: 0.199

The PEV of the unconstrained model (no zero loadings) is: 0.247

The `summary(model_selection_wst)` function displays the chosen cardinality and the proportion of explained variance (PEV) of the data associated with this level of cardinality, as well as the PEV of the unconstrained model (i.e., no zero loading). Figure 1 shows the evolution of PEV against the cardinality levels.

Analysis 2: Analysis of factor scores in Autism genetic data

As the `seafar` package was developed to specifically deal with high-dimensional setting, we also demonstrate how it can be used to analyze the Autism Genetic data set (Nishimura et al., 2007). The data is publicly available at <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE7329>. The R code used to download and pre-process the data is shown in Appendix B. The pre-processing involved removing three individuals who were mislabeled and standardizing each column of the data to have a mean of zero and a variance of one (as done in Guerra-Urzola et al., 2023). The resulting data set contains 43893 genetic marker variables for 27 subjects, in which 14 individuals are in the control group, six are in the FMR1-FM autism group, and seven are in the dup15q autism group. We chose three factors as indicated in the original paper by Nishimura et al. (2007). Due to the size of the data set, we perform a model selection with warm-start to determine the number of nonzero loadings from 200 candidate values:

Figure 1*Index of sparseness (IS) and proportion of explained variance (PEV) against cardinality level*

```
library(seafar)
data("autism")
```

```
autism_cardinality <- is.seafar_wst(
  data = autism,
  nfactors = 3,
  INIT = "rational",
  orthogonal = TRUE,
  card.length = 200
)
selcard <- autism_cardinality$selcard
summary(autism_cardinality)
```

The cardinality selected by the Index of Sparseness is: 37062

The PEV with this cardinality is: 0.251

The PEV of the unconstrained model (no zero loadings) is: 0.321

As can be seen, the chosen cardinality is 37062 nonzero loadings, which is less than a third of the total number of loadings (i.e., 43893×3 number of loadings). Using this value of cardinality, we run a SEAFa analysis with multiple starting values

```
autism_multistart <- seafar_multistart(  
  data = autism,  
  nfactors = 3,  
  C = selcard,  
  orthogonal = TRUE,  
  nstarts = 25  
)
```

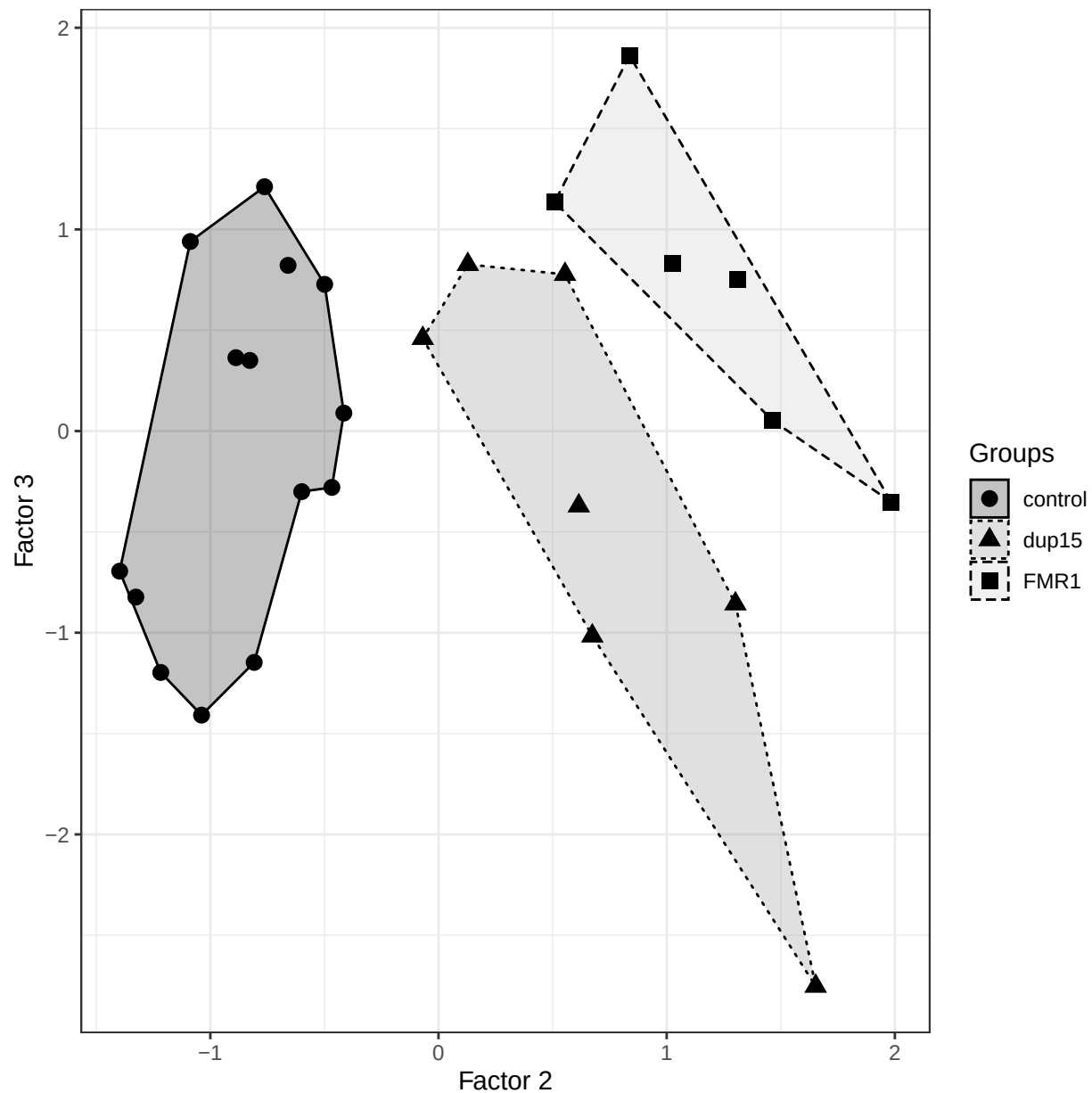
In this analysis, the factor scores were extracted for subsequent analysis, by using command `autism_multistart$scores`. Figure 2 shows a plot of Factor 2 and Factor 3 plotted against each other. As can be seen, Factor 2 clearly separated the control group and the two autistic groups. This reflects the same results by Nishimura et al. (2007), without having to use a filtering step (i.e., Nishimura et al. (2007) conducted an ANOVA to select a subset of 293 probes that maximally discriminated between the three groups, which were then used in a PCA).

Discussion and Conclusions

In this tutorial, we demonstrated how to perform exploratory factor analysis for complex models and high-dimensional data using the R package `seafar`. The underlying algorithm builds on the approximate factor model and extends the recently proposed Regularized ESEM approach (Le et al., 2026). In particular, `seafar` employs a cardinality constraint to impose simple structure on the loading matrix for both orthogonal and non-orthogonal factors. This constraint offers several advantages. First, it yields exact zero loadings, in contrast to rotation-based methods that rely on post hoc thresholding of small but nonzero values. Second, it provides direct control over the number of nonzero loadings, whereas with the LASSO penalty, for instance, users do not know beforehand how many (non)zero loadings are associated with which level of penalty.

talk about limitation of sorting big matrix and maybe the LASSO still needed.

The `seafar` package provides a user-friendly implementation of sparse exploratory factor

Figure 2*Factor scores plot*

analysis with minimal input requirements, along with built-in model selection procedures based on the Index of Sparseness. These procedures, suggested by RegularizedSCA (Gu et al., 2019) and Regularized ESEM (Le et al., 2026), allow users to determine the appropriate number of (non)zero loadings in the absence of strong prior theoretical guidance. The proposed package can be extended to accommodate categorical variables, further broadening its applicability.

References

- Adachi, K., & Kiers, H. A. (2017). Sparse regression without using a penalty function.
http://www.jfssa.jp/taikai/2017/table/program_detail/pdf/1-50/10009.pdf
- Blighe, K., Andrews, J., & Lun, A. (2026). PCAtools: PCAtools: Everything principal components analysis. <https://bioconductor.org/packages/PCAtools>
- Cattell, R. B. (1966). The scree test for the number of factors. Multivariate Behavioral Research, 1(2), 245–276. https://doi.org/10.1207/s15327906mbr0102_10
- Ding, C., & He, X. (2004). K-means clustering via principal component analysis. Proceedings of the Twenty-First International Conference on Machine Learning, 29.
- Epskamp, S., Cramer, A. O., Waldorp, L. J., Schmittmann, V. D., & Borsboom, D. (2012). Qgraph: Network visualizations of relationships in psychometric data. Journal of Statistical Software, 48, 1–18. <https://doi.org/10.18637/jss.v048.i04>
- Friedman, J. H., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. Journal of Statistical Software, 33(1), 1–22.
<https://doi.org/10.18637/jss.v033.i01>
- Gajjar, S., Kulahci, M., & Palazoglu, A. (2017). Selection of non-zero loadings in sparse principal component analysis. Chemometrics and Intelligent Laboratory Systems, 162, 160–171.
<https://doi.org/10.1016/j.chemolab.2017.01.018>
- Goretzko, D., & Bühner, M. (2020). One model to rule them all? Using machine learning algorithms to determine the number of factors in exploratory factor analysis. Psychological Methods, 25(6), 776. <https://doi.org/10.1037/met0000262>
- Gu, Z., Schipper, N. C. de, & Van Deun, K. (2019). Variable selection in the regularized simultaneous component analysis method for multi-source data integration. Scientific Reports, 9(1), 18608. <https://doi.org/10.1038/s41598-019-54673-2>
- Guerra-Urzola, R., Schipper, N. C. de, Tonne, A., Sijtsma, K., Vera, J. C., & Van Deun, K. (2023). Sparsifying the least-squares approach to PCA: Comparison of lasso and cardinality constraint. Advances in Data Analysis and Classification, 17(1), 269–286.

<https://doi.org/10.1007/s11634-022-00499-2>

Horn, J. L. (1965). A rationale and test for the number of factors in factor analysis.

Psychometrika, 30, 179–185. <https://doi.org/10.1007/BF02289447>

Kaiser, H. F. (1960). The application of electronic computers to factor analysis. Educational and Psychological Measurement, 20(1), 141–151.

<https://doi.org/10.1177/001316446002000116>

Le, T. T., Vermunt, J. K., Ballhausen, N., & Van Deun, K. (2026). Exploratory structural equation modeling and the curse of dimensionality. Behavior Research Methods, 58(3), 84.

<https://doi.org/10.3758/s13428-026-02960-y>

Nishimura, Y., Martin, C. L., Vazquez-Lopez, A., Spence, S. J., Alvarez-Retuerto, A. I., Sigman, M., Steindler, C., Pellegrini, S., Schanen, N. C., Warren, S. T., et al. (2007). Genome-wide expression profiling of lymphoblastoid cell lines distinguishes different forms of autism and reveals shared pathways. Human Molecular Genetics, 16(14), 1682–1698.

<https://doi.org/doi.org/10.1093/hmg/ddm116>

O'Connor, B. P. (2026). EFA.dimensions: Exploratory factor analysis functions for assessing dimensionality. <https://doi.org/10.32614/CRAN.package.EFA.dimensions>

Trendafilov, N. T. (2014). From simple structure to sparse components: A review. Computational Statistics, 29, 431–454.

William Revelle. (2026). Psych: Procedures for psychological, psychometric, and personality research. Northwestern University. <https://CRAN.R-project.org/package=psych>

Appendix A

Mathematical derivations for SEAFa algorithm

Here, we present the mathematical derivations for the update of factor scores and factor loadings for SEAFa.

1. Update of the factor scores conditional upon fixed values for the loadings is based on the following objective:

$$\begin{aligned} \min_{\eta_{11}, \dots, \eta_{iQ}, \dots, \eta_{NQ}} & \sum_{j=1}^J \sum_{i=1}^N \left(y_{ij} - \sum_q \eta_{iq} p_{jq} \right)^2 + \lambda \sum_{j=1}^J \sum_{q=1}^Q |p_{jq}|_1 \\ \text{subject to} & \sum_i \eta_{iq}^2 = N \quad \forall q = 1, \dots, Q. \end{aligned} \quad (\text{A1})$$

(A1) can be rewritten in matrix form as follows:

$$\begin{aligned} \min_{\eta_1, \dots, \eta_q, \dots, \eta_Q} & \|\mathbf{Y} - \sum_q \eta_q \mathbf{p}_q^T\|^2 + \sum_q \lambda |\mathbf{p}_q|_1 \\ \text{subject to} & \eta_q^T \eta_q = N \quad \forall q = 1, \dots, Q. \end{aligned} \quad (\text{A2})$$

Using the method of Lagrange Multipliers, we obtain the Lagrangian function

$$\begin{aligned} \mathcal{L} &= \|\mathbf{Y} - \sum_q \eta_q \mathbf{p}_q^T\|^2 - \mu (\eta_q^T \eta_q - N) \\ &= \|\mathbf{Y} - \sum_{t \neq q} \eta_t \mathbf{p}_t^T - \eta_q \mathbf{p}_q^T\|^2 - \mu (\eta_q^T \eta_q - N) \\ &= \|\mathbf{E}_q - \eta_q \mathbf{p}_q^T\|^2 - \mu (\eta_q^T \eta_q - N) \end{aligned} \quad (\text{A3})$$

Solving $\frac{\partial \mathcal{L}}{\partial \eta_q} = 0$ and $\frac{\partial \mathcal{L}}{\partial \mu} = 0$, we have $\eta_q = \frac{\sqrt{N} \mathbf{E}_q \mathbf{p}_q}{\sqrt{(\mathbf{E}_q \mathbf{p}_q)^T (\mathbf{E}_q \mathbf{p}_q)}}$.

2. The update of factor loadings conditional upon fixed values for the scores is based on the

following objective:

$$\begin{aligned} \min_{\mathbf{P}} & ||\mathbf{Y} - \mathbf{H}\mathbf{P}^T||_2^2 \\ \text{subject to } & \text{Card}(\mathbf{P}) = C, \end{aligned} \quad (\text{A4})$$

which can be rewritten in vectorized form as follows:

$$\begin{aligned} \min_{\mathbf{P}} & ||\text{vec}(\mathbf{Y}) - (\mathbf{I} \otimes \mathbf{H})\text{vec}(\mathbf{P}^T)||_2^2, \\ \text{subject to } & \text{Card}(\mathbf{P}) = C. \end{aligned} \quad (\text{A5})$$

As pointed out by (Adachi & Kiers, 2017), (A5) can be majorized as follows:

$$||\text{vec}(\mathbf{Y}) - (\mathbf{I} \otimes \mathbf{H})\text{vec}(\mathbf{P}^T)||_2^2 \leq c + \alpha ||\mathbf{b} - \text{vec}(\mathbf{P})||_2^2, \quad (\text{A6})$$

where the right-hand side function is also known as the surrogate function. Here, c is a constant with respect to $\mathbf{P}$, α is the maximum eigenvalue of $\mathbf{H}^T\mathbf{H}$. $\mathbf{b}$ is derived as follows:

$$\begin{aligned} \mathbf{b} &= \text{vec}(\mathbf{P}^T) - \frac{1}{\alpha} \left((\mathbf{I} \otimes \mathbf{H})^T (\mathbf{I} \otimes \mathbf{H}) \text{vec}(\mathbf{P}^T) - (\mathbf{I} \otimes \mathbf{H})^T \text{vec}(\mathbf{Y}) \right) \\ &= \text{vec}(\mathbf{P}^T) - \frac{1}{\alpha} \left((\mathbf{I} \otimes \mathbf{H}^T) (\mathbf{I} \otimes \mathbf{H}) \text{vec}(\mathbf{P}^T) - (\mathbf{I} \otimes \mathbf{H}^T) \text{vec}(\mathbf{Y}) \right) \\ &= \text{vec}(\mathbf{P}^T) - \frac{1}{\alpha} \left(\mathbf{I} \otimes (\mathbf{H}^T \mathbf{H}) \text{vec}(\mathbf{P}^T) - \text{vec}(\mathbf{H}^T \mathbf{Y}) \right) \\ &= \text{vec}(\mathbf{P}^T) - \frac{1}{\alpha} \left(\text{vec}(\mathbf{H}^T \mathbf{H} \mathbf{P}^T) - \text{vec}(\mathbf{H}^T \mathbf{Y}) \right) \\ &= \text{vec} \left(\mathbf{P}^T - \frac{1}{\alpha} (\mathbf{H}^T \mathbf{H} \mathbf{P}^T - \mathbf{H}^T \mathbf{Y}) \right), \end{aligned}$$

or in matrix form as

$$\mathbf{B} = \mathbf{P} - \frac{1}{\alpha} (\mathbf{P} \mathbf{H}^T \mathbf{H} - \mathbf{Y}^T \mathbf{H}), \quad (\text{A7})$$

where $b = \text{vec}(\mathbf{B}^T)$. Let $k = J \times Q - C$ be the number of zero loadings in $\mathbf{P}$ and $|b|_{(k)}$ is the

k -th smallest element of $\mathbf{B}$. Then the update of $\mathbf{P} = (p_{jq})$ is

$$p_{jq} = \begin{cases} b_{jq}, & \text{if } |b_{jq}| \leq |b|_{(k)}, \\ 0, & \text{otherwise} \end{cases} \quad (\text{A8})$$

Appendix B

Obtaining the Autism Genetic Data

The original data set from Nishimura et al. (2007) is publicly available and can be downloaded from <https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE7329>. We followed the steps done by Guerra-Urzola et al. (2023) to obtain and process the data prior to the analyses in this manuscript. The steps are as follows:

Step 1: Download the raw data set

```
data_link= 'https://ftp.ncbi.nlm.nih.gov/geo/series/GSE7nnn/GSE7329/matrix/GSE7329_series
destfile= paste0(getwd(), "/GSE7329_series_matrix.txt.gz")
download.file(data_link, destfile)
```

Step 2: Process data

The needed data was extracted from the full data set. Three individuals that were mislabeled were removed from the data set, and genetic markers with missing values were also removed. The data was then standardized and ready for analyses.

```
full_data=gzfile('GSE7329_series_matrix.txt.gz','rt')
full_data=read.csv(full_data,header=F,sep = '\t')

col_names= c()
for (i in 736:751) {
  col_names=c(col_names,sapply(full_data[i,], as.character))
}

# extract data needed for the analysis
matrix1= full_data[752:703647,]
matrix2= matrix(rep(0, prod(dim(matrix1)))), ncol = dim(matrix1)[2])
for(i in 1:nrow(matrix1) ){
  matrix2[i,]= as.numeric(sapply(matrix1[i,],as.character))
}

# put extracted data in a matrix with column names
Data_Autism=matrix(data = t(matrix2),byrow = TRUE, ncol = length(col_names))
colnames(Data_Autism)=col_names
```

```
# remove first (index) column and last column (all NA)
Data_Autism=Data_Autism[,-c(1,dim(Data_Autism)[2])]

# remove mislabel individuals
Data_Autism=Data_Autism[,-c(12,13,27)]

# transpose data matrix so rows are subjects, columns are gene expression
Data_Autism= t(Data_Autism)

# remove NAs
Data_Autism = Data_Autism[,-which(colSums(Data_Autism)==0)]

# standardized data
Data_Autism=scale(Data_Autism, center = TRUE, scale = TRUE)
Data_Autism=as.data.frame(Data_Autism)
```